

Using the Simple(r) Communication System

This document describes how to use the simplified multi-channel Type 630 Data Communication System replacement.

What is DCS2?

The original DCS was a multichannel serial I/O system for the PDP-1, -4, -5 and -6. It supported from 8 to 64 serial connections to, e.g. teletypes or modems. There were multiple variations covering the number of channels, the transmission speed, full or half duplex, etc. However, this really isn't particularly useful in today's world.

This is a simplified version of the new and improved DCS2 which uses sockets instead of serial connections to allow communicating with any device that has sockets, or any device that can open a remote socket. It is full duplex with separate send and receive buffers and control and can be configured at compile time for up to 64 independent channels.

The instructions are the same codes as the original and with very similar function, although with extensions to manage the sockets associated with each channel.

Unlike the more advanced and totally incompatible SCS version, DCS2 provides only single-character operations.

The original DCS allowed or-ing 02000 with the IOT to cause the IO register to be cleared before new data was added. and the behavior is maintained for rch and rcr. This allows multi-character words to be efficiently handled, as with the macro assembler (flexo directive. Only the low 6 bits for Flexo mode or 8 bits are used for the commands.

For all others the IO register is cleared if data is going to be returned in it.

DCS2 is implemented as a dynamic IOT, IOT_22 and requires no changes to the emulator as long as it has the Dynamic IOT extension. Just copy IOT_22.so to the IOTs directory, that's it, not even a restart is needed.

How does it work?

In order to use one of the 'channels', an IOT must be used to tell DCS2 how to operate, such as server expecting a connection on the local socket, client opening a connection on a remote socket. The configuration also specifies the DCS2 channel to assign and other information. The IOT passes the address of a configuration block in local memory. Bytes can then be sent and received via the IOT commands below.

The original and this version implement a 'scanner' that looks at incoming connections to find one that has input ready to read. A single channel is assigned as the current channel and it maintains control until it releases control, which then allows the scanner to select a new channel. It does this in a round-robin fashion so one channel can't monopolize control. If the current channel is locked, then the ready channel is just marked as ready to read and becomes a candidate for selection.

For sending, either the current channel can be used or a different channel can be specified. This does not affect the scanning algorithm, an explicit send channel selection remains valid until changed.

An explicit channel selection will automatically unlock the current channel and lock the new one. Again, it must be released before scanning will continue from the next channel after.

Internally the IOT uses epoll() to maximize performance and reduce overhead. No channel will be selected by the

'scanner' unless it has data to read. It is assumed that data can always be sent, but error are possible and can be detected.

Client vs server mode

Each channel can be either a TCP/IP client or server.

For server mode, the channel listens on the specified port for a connection request. A check is made for a request every 100us. When a connection is established, the status of the channel will indicate so. Only one remote system can connect to a channel and a channel will not be selected by the scanner until a channel is connected.

Multiple channels can have the same port in which case they act as a connection pool. When a request comes in, the first channel in listening state with the requested port will be assigned to that client.

A channel can be closed to terminate the connection, or the connection itself can be closed by the client. If the connection is closed, a server channel will return to listening for a connection. A client channel will be closed and can only be used by opening it again.

For client mode, when the channel is opened, it tries to connect to the specified host. In order to not halt the system while waiting, a non-blocking connect is used. As with server mode, it will be polled every 100us. When it is established, the channel becomes a candidate for selection by the scanner and can be used by the send commands.

Flexo, FIODEC, Consise?

There are two character sets, Concise and FIODEC. They are virtually identical except FIODEC has bit 0200 set as a parity bit. It is set to ensure an odd number of 1 bits in the character.

To convert from/to Concise to/from FIODEC:

```
fio = (con & 1)?con:(con | 0200);  
con = fio & 077;
```

So, what is 'flexo'? It's just Concise code, that's it. This is what the macro directives char, flexo, and text produce.

During ascii to flexo conversion, some characters cannot be represented. In this case, the special character flxnch, 013, will be returned. The full list of special characters is:

- flxnch 013 no flexo equivalent for ascii character
- flxetx 013 end marker for some string operations
- flxerr 076 error marker, some error happened in rcr or rch
- flxnl 077 the end-of-line character, flexo carriage return
- ascnch 077 a flexo shift or unshift character was seen by rxl in flexo to ascii mode

Carriage return, linefeed, newline, echo?

This is an area on which there is plenty of variation. Some systems indicate end-of-line with a linefeed character, aka newline, ascii 012. Some use carriage return, 015, followed by linefeed. Some just use carriage return. DCS2 has configurable behavior, but the behavior depends upon the flexo mode.

If echo mode is on, the incoming character is immediately echoed back to the remote host with no modification. If it's off, then no echoing is done.

If flexo mode is on, the rules are simple. Concise/flexo does not have a linefeed character, only carriage return, 077. If crnl is not on, then this will be converted to the newline character on output.

A carriage return on input will be converted to flexo 077. A newline will be ignored. If crnl is on, then sending a flexo carriage return will result in sending an ascii carriage return followed by an ascii newline.

If flexo mode is off, then the rules are a bit different. If crnl is on, then sending an ascii carriage return will result in a carriage return followed by a newline being sent. Sending a newline does not have any special meaning, it is sent as it is.

For crnl on or off, the incoming characters are passed unchanged, both carriage return and newline.

Notes on waits, completion pulses, and channel starvation

Only one command, rwe, below, will honor either the wait bit (i) or the completion pulse request. All others will always immediately return.

Why? Remember that this is an asynchronous socket communications layer. Blocking in a wait could mean that other active channels you have opened would not get serviced until the current wait completed. This is not good for the socket world, you could lose data.

So why does rwe wait? Because it isn't waiting on one channel, it responds to any event on any channel and so can't result in a service blocking wait.

If you want to not have to check busy flags constantly, you should use rwe or interrupts to know when some socket event has occurred.

It's also a very bad idea to pay attention to just one channel, assuming you have multiple channels open. The primary commands use the current channel and must unlock it for another channel to be automatically selected. You can also use the roc command, below, to override channel the current channel and force a new channel to be the active, locked one.

Errors

For most commands, the IO register will contain an error status on return. If bit 0 is 1, an error occurred and will be in bits 14-17. If bit 1 is also 1, the Linux errno is included in bits 6-13. See below for the DCS2 error codes.

The IOT commands

Note: The current memory bank is honored in all calls that are passed 12-bit addresses, so DCS2 can be used in extended memory mode.

Warning: The mnemonics below are directly from the DEC documentation, but notice there is an *rcr* instruction. That conflicts with the *rcr* that is rotate-combined-right. Keep that in mind. The include files provided with m1pp use *rchr* instead.

All are implemented via a single IOT, 22 as mentioned. The following IOT commands are supported:

- rch 720022 receive a character from the current receive channel into the IO register
- rcr 721022 same as rch except also release the channel, combines rch and rsc
- tcb 724022 send a character from the IO register to the current send channel
- tcc 725022 send a character from the IO register to the current receive channel and release the channel
- rrc 720122 get the active channel number into IO register bits 12-17
- rsc 721122 release the current receive channel
- ssb 724122 select a new send channel, IO register bits 12-17 select the channel

Note again that setting bit 02000 on rch and rcr will clear the IO register before reading the next character. All other commands except rwe always change the IO register on return.

No operations will succeed until at least one channel has been opened via scb, below.

Flexo mode is honored for rch, rcr, tcb, and tcc. The characters will be converted to ascii before sending, or converted from ascii on reading. See the detailed description in the Channel Control Block description below.

Rcr, rch, tcb, and tcc use the low 6 bits in flexo mode, else the low 8 bits.

For rch and rcr, the action on an error depends upon if flexo mode is enabled. For flexo, for no character ready the low 6 bits returned will be flxnch. For any error, the low 6 bits returned will be flxerr.

For 8-bit mode, if no character is available IO register bits 0 and 1 will both be 1 and the other bits unchanged. If any other error occurs, bit 0 will be set, bit 1 cleared, and the rest of the register set to the full error code.

The tcb and tcc commands will fail if the channel isn't open and connected, the transmit buffer is full, or there is a socket error. A socket error is most likely because the remote end isn't listening any more.

if bits 0 and 2 are set on return, the send buffer is full, the other bits will be left unchanged. When space is again available, the channel status will be updated. If any other error occurs, bit 0 will be set, bit 1 cleared, and the rest of the register set to the full error code.

If no errors occurred for tcb and tcc, the IO register will be cleared.

If Flexo is enabled, upper and lower shift characters do not get sent, they are just state changes. Don't count them as sent characters. Any Concise character that has no ascii mapping will be ignored.

The last error on a channel is saved and can be fetched via the rle command. A successful operation on a channel clears the last error.

If echo is enabled, the input character *as received* will immediately be sent back.

For rrc, if there is no active channel, that is, no channel is being held, on return IO register bit 0 will be a 1 and the NOCHANNEL error will be set.

And the new extended commands:

- scb 724222 set/clear/rebind channel request

On call:

IO register bit 5 set to 1 is a set request

IO register bits 6-17 are the address of the request block in memory

IO register bit 4 set to 1 is a rebind request
IO register bits 12-17 are the channel number to rebind

Rebind only applies to server channels. This closes any currently open socket on the channel and returns to listen mode.

IO register bit 3 set to 1 is a reset request, no other bits used

A complete reset of DCS2 is done. A reset will close all open channels as well as clear internal state and disable polling resetting DCS2 to its startup state.

If neither set, reset, or rebind is set, this is a clear request.

IO register bits 12-17 are the channel number to clear

The channel's ports will be closed, it will have its open flag cleared, and it will be available for setting again.

A channel set as a server will listen on the assigned port for a connection request. When a connection is established, the channel status will indicate so.

A channel set as a client will attempt to open the socket and port specified in the request block. When the connect succeeds, the channel status will indicate so.

A clear on either type of channel will close any open connection and mark the channel as not in use.

On a reset, all open channels will be closed, polling stopped, and the DCS2 system reset to inactive. A subsequent channel set request will reinitialize it. A reset has no request block, any address is ignored.

On return from any of the above, the IO register will be cleared for success. On an error, bit 0 will be set and the error code returned.

- rle 724322 get last error

On return the IO register will contain the full error word of the last error seen

This is useful for getting error codes that aren't returned as part of a command return.

- rpc 724422 receive pending count

On call, IO register bits 12-17 should contain the channel number to check.
On return the IO register will contain the number of bytes currently waiting to be read.

- rci 724522 clear and reset interrupt status

On call, IO register bits 12-17 should contain the channel number to use.
On return, the IO register will be unchanged unless the channel number is invalid in which case bit 0 will be 1 and a channel out of bounds error set.

This also reenables interrupts for the channel, but only if interrupts were allowed when the channel was opened. It should only be called if you are actually returning from an interrupt, or you can lose pending events.

- ric 724622 get interrupting channel

On return, the IO register will contain the last channel number that caused an interrupt or 0100, an invalid channel, if none.

- rcs 724722 get the status for a channel

On call, IO register bits 12-17 should contain the channel number to use.

On return, the IO register will contain the the full status for the channel.

See the section on channel status for details.

If an invalid channel number is passed, the invalid channel status will be returned.

- rwe i 735122 wait for event

There is no input or output, the IO register will be unchanged.

This is the only command that will block, and it will always block. Why? Because any event on any channel will release the wait and so avoid starving channels, no wait can occur on a single channel.

It will resume if a connection is established, a channel has a character ready to read, a previously-full transmit buffer becomes available to write, the remote end closed a connection, or for any socket error.

If the wait bit is not set, op code 725122, it will return immediately.

- roc 725222 override current channel

On call, IO register bits 12-17 should contain the channel number to use.

On return, the IO register will be 0 unless:

If an invalid channel number is passed, the invalid channel status will be returned.

If the channel requested is not open, the not open status will be returned.

The new channel must be open. The current channel, if any, is unlocked. The new channel becomes the current channel and will be locked. If an error occurred, the current channel and lock state will be unchanged.

- res 725322 enable/disable sbs16

On call, IO register bit 17 set to 1 enables sbs16, else disables it.

On return, the IO register will have the prior setting.

- rxl 725422 convert between flexo and ascii

On call:

IO register bit 0 set to 1 is flex to ascii, otherwise ascii to flex

IO register bit 9 set to 1 means upper-shift for flex, see note

IO register bits 10-17 are the character to convert

On return:

IO register bit 0 will be unchanged

IO register bit 8 set to 1 for ascii to flex means the flex shift state changed

IO register bit 9 set to 1 for ascii to flex means in upper shift state

IO register bits 10-17 are the converted character

This is a convenience IOT that will convert back and forth between flexo/concise and ascii. It uses the same mappings as the regular send and receive translations.

The interaction of bits 8 and 9 are important.

Bit 9 is the current shift state, set to a 1 if currently in upper shift mode, else 0. Typically, this would initially be set to 0.

IMPORTANT, after the initial setting, the application should not change this bit until the current character stream is completed. Otherwise, a valid character might never be returned. When a shift, upper or lower, occurs, bit 9 will be updated to reflect the shift state and bit 8 set to indicate a shift change happened.

For ascii to flex conversion, the character returned will be the appropriate shift character. If there is no equivalent flex character for an ascii character, flxnch will be returned.

For flexo to ascii conversion, a shift character will be ignored and the returned character will be ascnch.

For either mode, the application should resubmit the character if bit 9 is a 1. It can also check for ascnch in flexo to ascii mode or an upper or lower shift character in ascii to flexo mode.

The Channel Request Block

The request block varies depending upon whether the channel is a server or a client. A set for a server requires 2 words, a client 4 words. A clear, rebind, or reset use no words, just the IO register contents.

Flags in word 0 of the request block for a set:

nfECerissssmccccc

where:

bit 0, n	convert carriage return to carriage return and a linefeed on output This is the telnet standard, and note that the Concise character set does not have linefeed so it will be dropped on input when Flexo mode is on.
bit 1, f	do Flexo conversion; outgoing characters are converted to ascii from Flexo, the reverse for incoming ones, 0 to just pass the data unchanged as 8 bit binary
bit 2, E	echo input
bit 3, C	interrupt on connecton established or lost
bit 4, e	interrupt on a socket error other than established or lost
bit 5, r	interrupt on recieved characters available
bit 6, i	1 to allow interrupts
bits 7-10, ssss	if i is 1, the SBS channel to use to interrupt, always treated as 0 if no SBS16
bit 11, m	1 for server, 0 for client
bits 12-17, ccccc	the channel number up to 63 depending upon configuration

Enabling interrupts here immediately makes the channel a candidate for interrupts, so be sure you have set up your interrupt locations, etc. that the sbs interrupt system will use, and then enable it via the esm IOT.

For a server channel:

Word 1 bits 2-17 specify the port number to listen on

For a client channel:

Word 1 bits 2-17 specify the port number to connect to

Word 2 bits 2-17 specify the high 16 bits of the IP address to connect to

Word 3 bits 2-17 specify the low 16 bits of the IP address to connect to

Thus, an ip address of 10.20.30.40 would have word 2 bits 2-8 set to 10, 9-17 set to 20, word 3 bits 2-8 set to 30, word2 bits 9-17 set to 40. Remember, these are decimal values, the octal equivalents would be 012, 024, 036, 050.

Once a channel has been opened, the request block is no longer used and may be reused for other purposes. A channel can only be bound to one port at a time. For server mode, allocate multiple channels with the same port to allow multiple connections to that port. If there is no channel available to accept a connection request, the request will be processed when a channel becomes available for the associated port. Note that the client might abandon the request after some time limit.

Channel status bits

If a channel's status is fetched via rcs, the status bits are:

Bit 0	unused
Bit 1	unused
Bit 2	unused
Bit 3	unused
Bit 4	unused
Bit 5	unused
Bit 6	is the current channel
Bit 7	interrupted because of a socket error or connection change
Bit 8	interrupted because the transmit buffer is full
Bit 9	interrupted because character received
Bit 10	interrupts enabled
Bit 11	remote closed connection
Bit 12	has a socket error
Bit 13	has pending characters to read
Bit 14	send buffer is full and can't take more characters
Bit 15	is connected and ready for use
Bit 16	is a server
Bit 17	is open

When an interrupt occurs, bits 7-9 can be used to determine why. For connects or disconnects, bits 15 and 11 will indicate the state.

Check Status, cks, bits

If check status is executed, IO register bit 10 will be 1 if there is any pending DCS2 error.

Error flags

These error flags can be returned in the IO register and also in the last error reported by rle and rcs. See the individual commands for more details.

Bit 0	1 means an error occurred
Bit 1	1 means the Linux errno is included
Bit 2	
Bit 3	
Bit 4	
Bit 5	sbs err
Bit 6	sbs err
Bit 7	sbs err
Bit 8	sbs err
Bit 9	always 0
Bit 10	errno
Bit 11	errno
Bit 12	errno
Bit 13	errno
Bit 14	errno
Bit 15	errno
Bit 16	errno
Bit 17	errno

If the Linux errno is included, it will be in bits 10-17.

The DCS2 errors are in bits 5-8:

01 - not open, a request is being made on a channel that is not currently open
02 - already open, an scb set was done on an already open channel
03 - illegal channel, a number greater than the number of allowed channels has been used
04 - not server, a scb rebind command is being made on a non-server channel
05 - invalid command, an IOT xx22 was done but there is no such command
06 - socket error, an open or other operation on a socket failed
07 - bind error, a bind() on a port failed
10 - no current channel, no channel is current and a current-channel command was issued
11 - connection was lost, the socket is no longer connected, probably a client close
12 - epoll error during polling
13 - no remote connection established but a write, tcc or tcb, was done

These are the octal values.

Interrupts aka Sequence Breaks

The DCS2 system can use the SBS or SBS16 system by specifying such in the request block. If only SBS is available, the channel number is ignored and is always 0. On an interrupt, ric can be used to see which channel, if any, needs attention.

An interrupt can occur if any error condition is raised, a connection is established or lost, if any bytes are ready to be read, or a transmit buffer is full, controlled by the settings specified when the channel was opened.

If the SBS16 system is in use, different SBS channels can be assigned to each DCS2 channel for convenience.